

COMPTE RENDU

Administration Réseaux et Programmation Système - TP1 - C-Shell &
Processus

3e année Cybersécurité - École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha

Filière : Cybersecurité

Année : 2025/2026

Enseignante : M.Noufel & M.ElAmrani

Date : 21 février 2026

Table des matières

1	Exercice 1 : La différence entre un processus et une fenêtre	2
1.1	Affichage initial des processus (<code>ps</code>)	2
1.2	Lancement de <code>gedit &</code> et observation du PID	2
1.3	Vérification : <code>gedit</code> apparaît dans <code>ps</code>	2
1.4	Saisie d'un texte dans <code>gedit</code> et sauvegarde dans <code>toto</code>	3
1.5	Ouverture d'une nouvelle fenêtre <code>gedit</code> et observation de <code>ps</code>	3
1.6	Lancement d'un second <code>gedit</code> depuis le terminal	3
1.7	Remarque — Processus vs fenêtre : synthèse	4
2	Exercice 2 : Utilisation de <code>kill</code>	4
2.1	Lancement de <code>less ocs.txt</code> dans le Terminal 1	4
2.2	Identification du PID de <code>less</code> depuis le Terminal 2	5
2.3	Arrêt de <code>less</code> avec <code>kill (SIGTERM)</code>	5
2.4	Comparaison : <code>kill</code> vs <code>kill -9 (SIGKILL)</code>	5
2.5	Interruption d'un programme avec <code>Ctrl+C</code>	6
3	Exercice 3 : Redirection des Entrées Sorties	6
3.1	Création et affichage de <code>fich1</code> et <code>fich2</code>	6
3.2	Concaténation de <code>fich1</code> et <code>fich2</code> dans <code>resu1</code>	6
3.3	Tri d'un fichier avec <code>sort</code>	6
3.4	Utilisation de <code>wc</code> en entrée standard	7
3.5	Comptage des mots de <code>ocs.txt</code> avec <code>wc -w</code>	7
3.6	Redirection du résultat de <code>wc -w</code> dans <code>resu2</code>	8
4	Exercice 4 : Les pipes	8
4.1	Affichage du répertoire trié par taille avec <code>ls -l sort -n -k 5</code>	8
4.2	Comptage des objets du répertoire avec <code>ls wc -w</code>	9
5	Exercice 5 : Tâches de fond	9
5.1	Lancement de <code>gedit</code> sans <code>&</code> : blocage du terminal	9
5.2	Suspension avec <code>Ctrl+Z</code> , reprise avec <code>bg</code> et <code>fg</code>	9
5.3	Lancement direct avec <code>gedit &</code>	11
6	Exercice 6 : Alias	11
6.1	Création d'un alias temporaire	12
6.2	Alias permanents via <code>~/.bashrc</code>	12
7	Exercice 7 : Variable <code>PATH</code>	13
7.1	Affichage de <code>\$PATH</code> et localisation de commandes avec <code>type</code>	13
7.2	Ajout d'une variable dans <code>~/.bashrc</code>	13
	Conclusion	14

1 Exercice 1 : La différence entre un processus et une fenêtre

Un processus est un programme en cours d'exécution, identifié de manière unique par son **PID** (*Process IDentifier*). La commande `ps` permet de visualiser les processus actifs du terminal courant, tandis que `ps -u` ou `ps aux` liste tous les processus de l'utilisateur (ou du système entier).

1.1 Affichage initial des processus (ps)

Avant de lancer quoi que ce soit, on observe les processus du terminal courant :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ ps
  PID TTY          TIME CMD
 44150 pts/4        00:00:00 bash
 45057 pts/4        00:00:00 ps
```

FIGURE 1 – État initial : seuls `bash` et `ps` sont présents (commande `ps`)

On constate que le terminal ne contient que le shell `bash` (PID 45425) et la commande `ps` elle-même (PID 45432), ce qui est l'état minimal d'un terminal actif.

1.2 Lancement de `gedit` & et observation du PID

On lance `gedit` en arrière-plan avec l'opérateur `&` afin de conserver la main dans le terminal :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ gedit &
[1] 45204
```

FIGURE 2 – Lancement de `gedit &` : le shell affiche le numéro de job `[1]` et le PID 45204

Le shell affiche `[1] 45204` : `[1]` est le numéro de *job* local au shell, et 45204 est le PID global du processus `gedit` nouvellement créé.

1.3 Vérification : `gedit` apparaît dans `ps`

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ ps
  PID TTY          TIME CMD
 44150 pts/4        00:00:00 bash
 45204 pts/4        00:00:00 gedit
 45219 pts/4        00:00:00 ps
```

FIGURE 3 – Après `gedit &` : le processus `gedit` (PID 45204) est bien visible dans `ps`

On observe désormais trois processus sur ce terminal : `bash` (PID 44150), `gedit` (PID 45204) et `ps` (PID 45219). Le processus `gedit` est bien associé au même TTY (`pts/4`) que notre shell.

1.4 Saisie d'un texte dans gedit et sauvegarde dans toto

On tape un court texte dans la fenêtre `gedit` ouverte, puis on le sauvegarde dans un fichier nommé `ocs.txt` stocké dans un répertoire nommé `toto`. On peut vérifier que le fichier a bien été créé à l'aide de `ls -R` :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ ls -R .
.:
ex1  TP2.pdf

./ex1:
toto

./ex1/toto:
ocs.txt
```

FIGURE 4 – Vérification avec `ls -R .` : le fichier `ocs` est présent dans le sous-répertoire `ex1/toto/`

1.5 Ouverture d'une nouvelle fenêtre gedit et observation de ps

On ouvre un deuxième terminal, puis on y exécute `ps` :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ ps
  PID TTY          TIME CMD
 45425 pts/5        00:00:00 bash
 45432 pts/5        00:00:00 ps
```

FIGURE 5 – Depuis le Terminal 2 : `ps` n'affiche pas `gedit`, car celui-ci tourne sur un TTY différent (Terminal 1)

Explication : La commande `ps` sans option n'affiche que les processus associés au **TTY courant**. `gedit` ayant été lancé depuis le Terminal 1 (`pts/5`), il est invisible depuis le Terminal 2 (`pts/4`). Les shells sont donc **indépendants** pour la visibilité de leurs processus via `ps` simple. Pour voir tous les processus du système, il faut utiliser `ps aux` ou `ps -u`.

1.6 Lancement d'un second gedit depuis le terminal

On relance `gedit &` depuis le Terminal 1 :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ gedit &
[1] 46205
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ ps
  PID TTY          TIME CMD
 45425 pts/5        00:00:00 bash
 46205 pts/5        00:00:00 gedit
 46219 pts/5        00:00:00 ps
```

FIGURE 6 – Second `gedit &` depuis le Terminal 1 : un nouveau processus (PID 46205) apparaît dans `ps` aux côtés du premier

Cette fois, `ps` exécuté depuis le même terminal révèle un **nouveau processus** `gedit` avec un PID distinct. Son espace mémoire est totalement disjoint du premier : les deux instances s'exécutent de manière **indépendante**.

1.7 Remarque — Processus vs fenêtre : synthèse

- **Cas 1 — Nouvelle fenêtre via l'interface graphique** : les deux fenêtres sont gérées par **un seul processus**. Celui-ci connaît les informations de chacune des deux fenêtres. Il n'y a pas de nouveau PID.
- **Cas 2 — Nouveau `gedit` & depuis le terminal** : on crée un **nouveau processus** avec son propre PID. Son espace de données est disjoint du processus précédent ; les deux exécutions sont totalement indépendantes.

Point clé sur la portée des jobs : la commande `jobs` n'affiche que les tâches lancées depuis le *shell courant*. En revanche, `ps` (ou `ps aux`) liste tous les processus du système, indépendamment du terminal d'où ils ont été lancés. Chaque shell est donc **indépendant** pour la gestion de ses jobs, mais les processus eux-mêmes sont **globaux** au niveau du système d'exploitation.

2 Exercice 2 : Utilisation de `kill`

La commande `kill` permet d'envoyer un **signal** à un processus. Par défaut, elle envoie le signal `SIGTERM` (15), qui demande poliment au processus de se terminer. Le processus peut cependant choisir d'ignorer ce signal. L'option `-9` envoie `SIGKILL`, qui est un signal irrésistible : le noyau tue le processus de force, sans possibilité d'ignorance ni de nettoyage.

2.1 Lancement de `less ocs.txt` dans le Terminal 1

On lance la commande `less ocs.txt` dans un premier terminal. Le processus `less` prend le contrôle du terminal et affiche le contenu du fichier :

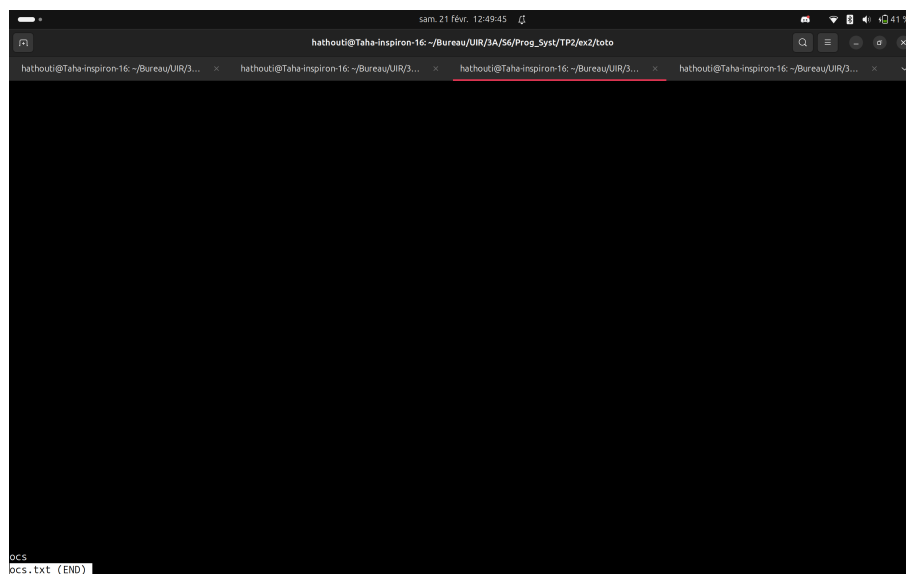


FIGURE 7 – Affichage de `ocs.txt` par `less` dans le Terminal 1

2.2 Identification du PID de less depuis le Terminal 2

Depuis un deuxième terminal, on exécute `ps -u` pour lister tous les processus de l'utilisateur. On repère ainsi le processus `less` avec son PID :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex2/toto$ ps -u
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
hathouti  3083  0.0  0.0 236088  6704 tty2    Sl+   févr.16   0:00 /usr/libexec/gdm-x-session --run-script env GNOME_SHELL_SESSION_MODE=ubuntu /usr/b
hathouti  3088  1.3  1.3 27014788 429196 tty2    Sl+   févr.16  92:58 /usr/lib/xorg/Xorg vt2 -displayfd 3 -auth /run/user/1000/gdm/Xauthority -nolisten
hathouti  3126  0.0  0.0 298528 17212 tty2    Sl+   févr.16   0:00 /usr/libexec/gnome-session-binary --session=ubuntu
hathouti  5177  0.0  0.0 11968  6512 pts/0    Ss+   févr.16   0:00 bash
hathouti  27902 0.0  0.0 12628  7048 pts/3    Ss+   févr.16   0:00 bash
hathouti  44150 0.0  0.0 13388  7804 pts/4    Ss   févr.17   0:00 bash
hathouti 171644 0.0  0.0  8624  2812 pts/4    S+   12:47   0:00 less ocs.txt
hathouti 171652 0.2  0.0 11244  5656 pts/1    Ss   12:47   0:00 bash
hathouti 171660 0.0  0.0 15324  5848 pts/1    R+   12:47   0:00 ps -u
```

FIGURE 8 – `ps -u` depuis le Terminal 2 : le processus `less ocs.txt` (PID 171644) est visible

On observe que `less` (PID 171644) tourne sur `pts/4` (Terminal 1), tandis que `ps -u` est exécuté depuis `pts/1` (Terminal 2). Cela confirme que `ps -u` liste les processus de tous les terminaux de l'utilisateur, contrairement à `ps` seul qui se limite au TTY courant.

2.3 Arrêt de less avec kill (SIGTERM)

On envoie le signal par défaut `SIGTERM` au processus `less` via son PID :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex2/toto$ kill 171644
```

FIGURE 9 – `kill 171644` : envoi du signal `SIGTERM` au processus `less`

Le processus `less` se termine et rend la main dans le Terminal 1 :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex2/toto$ less ocs.txt
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex2/toto$
```

FIGURE 10 – Terminal 1 : `less` s'est terminé proprement suite au `kill`

2.4 Comparaison : kill vs kill -9 (SIGKILL)

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex2/toto$ kill -9
kill : utilisation : kill [-s sigspec | -n signum | -sigspec] pid | jobspec ... ou kill -l [sigspec]
```

FIGURE 11 – Tentative de `kill -9` sans PID : le shell affiche le message d'utilisation de la commande

La différence entre les deux signaux est fondamentale :

- `kill PID` envoie **SIGTERM** (signal 15) : c'est une demande de terminaison que le processus peut intercepter, traiter ou même ignorer. Il permet au processus d'effectuer un nettoyage avant de mourir (fermeture de fichiers, libération mémoire, etc.);
- `kill -9 PID` envoie **SIGKILL** (signal 9) : ce signal est géré directement par le noyau et ne peut **pas** être ignoré. Le processus est tué immédiatement, sans aucun nettoyage possible. Du côté de `less`, le message affiché dans le terminal change : au lieu d'une terminaison propre, on obtient un message `Processus arrêté`.

2.5 Interruption d'un programme avec Ctrl+C

Le signal `SIGINT` (signal 2) est envoyé au processus au premier plan lorsque l'utilisateur appuie sur `Ctrl+C`. Contrairement à `SIGKILL`, ce signal peut être intercepté par le programme.

Remarque : le TP fait référence à un script *C-shell*. Or, l'environnement utilisé ici est `bash` et non `csh/tcsh`. Le comportement de `Ctrl+C` reste identique dans les deux shells (envoi de `SIGINT`), mais l'écriture d'un script C-shell n'était pas applicable dans ce contexte. Dans tous les cas, `Ctrl+C` interrompt bien tout processus en avant-plan qui ne bloque pas explicitement ce signal.

3 Exercice 3 : Redirection des Entrées Sorties

Un processus lit l'information sur l'entrée standard (clavier) et écrit sur la sortie standard (écran). Le shell permet de rediriger ces flux vers des fichiers grâce aux opérateurs suivants : `>` (redirection stdout), `>>` (append), `2>` (redirection stderr), `<` (redirection stdin).

3.1 Création et affichage de fich1 et fich2

On crée les deux fichiers en une seule ligne de commande avec `echo` et la redirection `>`, puis on vérifie leur contenu avec `more` :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$ echo "Une breve histoire du temps" > fich1 && echo "Stephen Hawking" > fich2
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$ more fich1 && more fich2
Une breve histoire du temps
Stephen Hawking
```

FIGURE 12 – Création de `fich1` et `fich2` puis affichage avec `more`

`fich1` contient *"Une breve histoire du temps"* et `fich2` contient *"Stephen Hawking"*.

3.2 Concaténation de fich1 et fich2 dans resu1

On utilise `cat` avec plusieurs arguments pour concaténer les deux fichiers et rediriger le résultat dans `resu1`, puis on vérifie avec `more` :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$ cat fich1 fich2 > resu1
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$ more resu1
Une breve histoire du temps
Stephen Hawking
```

FIGURE 13 – `cat fich1 fich2 > resu1` : concaténation et vérification avec `more resu1`

`resu1` contient bien la concaténation des deux fichiers, dans l'ordre spécifié.

3.3 Tri d'un fichier avec sort

On utilise `sort` pour trier les lignes de `resu1` alphabétiquement et rediriger le résultat dans `resu1.trie` :

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$ sort resu1 > resu1_trie
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$ more resu1
Une breve histoire du temps
Stephen Hawking
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$ more resu1_trie
Stephen Hawking
Une breve histoire du temps
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$

```

FIGURE 14 – `sort resu1 > resu1_trie` : `resu1_trie` contient les lignes triées alphabétiquement

On constate que "*Stephen Hawking*" précède désormais "*Une breve histoire du temps*" dans le fichier trié, conformément à l'ordre alphabétique.

3.4 Utilisation de `wc` en entrée standard

La commande `wc` permet de compter les caractères (`-c`), les mots (`-w`) et les lignes (`-l`). On teste d'abord ces options sur `resu1`, puis on utilise `wc -w` en mode interactif (saisie au clavier, fin avec `Ctrl+D`) :

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$ wc -c resu1
44 resu1
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$ wc -w resu1
7 resu1
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$ wc -l resu1
2 resu1
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$ wc -w
Une breve histoire du temps
Stephen Hawking
Une
Breve
Histoire
Du
Temps
Stephen
Hawking
14

```

FIGURE 15 – `wc -c`, `wc -w`, `wc -l` sur `resu1`, puis `wc -w` en mode interactif

`resu1` contient 44 caractères, 7 mots et 2 lignes. En mode interactif, après avoir saisi les mots un par un et terminé par `Ctrl+D`, `wc -w` retourne 14 mots.

3.5 Comptage des mots de `ocs.txt` avec `wc -w`

On copie le fichier `ocs.txt` depuis l'exercice 2, puis on utilise `wc -w` pour compter ses mots :

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$ cp ../ex2/toto/ocs.txt .
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$ cat ocs.txt
ocs
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$ wc -w ocs.txt
1 ocs.txt

```

FIGURE 16 – `wc -w ocs.txt` : le fichier contient 1 mot

Le fichier `ocs.txt` ne contient qu'un seul mot : "*ocs*".

3.6 Redirection du résultat de `wc -w` dans `resu2`

On redirige le résultat du comptage dans un fichier `resu2` et on vérifie son contenu avec `cat` :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$ wc -w ocs.txt > resu2
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$ cat resu2
1 ocs.txt
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex3$
```

FIGURE 17 – `wc -w ocs.txt > resu2` : vérification avec `cat resu2`

`resu2` contient bien 1 `ocs.txt`, ce qui correspond au résultat attendu du comptage de mots.

4 Exercice 4 : Les pipes

Un pipe (`|`) permet de brancher la sortie standard d'une commande sur l'entrée standard d'une autre, formant ainsi une chaîne de traitement. On crée d'abord trois fichiers de tailles différentes dans le répertoire `ex4` :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex4$ echo "court" > court.txt
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex4$ echo "Une Breve Histoire du Temps par Stephen Hawking" > moyen.txt
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex4$ echo "Et licet quocumque oculos flexeris feminas adfatin multas spectare cirratas, quibus, si nupsissent, per aetatem tam nixus poterat suppetere liberorum, ad usque taedium pedibus pavimenta tergentes tactari volucriter gyris, dum exprimunt innumera simulacra, quae finxere fabulae theatrales." > long.txt
```

FIGURE 18 – Création de trois fichiers de tailles différentes : `court.txt`, `moyen.txt` et `long.txt`

4.1 Affichage du répertoire trié par taille avec `ls -l | sort -n -k 5`

La commande `sort -n -k i` trie un tableau selon la valeur numérique croissante de la *i*-ème colonne. Dans la sortie de `ls -l`, la colonne 5 correspond à la taille en octets. On peut ainsi afficher les fichiers du plus petit au plus grand :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex4$ ls -l
total 12
-rw-rw-r-- 1 hathouti hathouti 6 févr. 21 14:12 court.txt
-rw-rw-r-- 1 hathouti hathouti 293 févr. 21 14:13 long.txt
-rw-rw-r-- 1 hathouti hathouti 48 févr. 21 14:12 moyen.txt
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex4$ ls -l | sort -n -k 5
total 12
-rw-rw-r-- 1 hathouti hathouti 6 févr. 21 14:12 court.txt
-rw-rw-r-- 1 hathouti hathouti 48 févr. 21 14:12 moyen.txt
-rw-rw-r-- 1 hathouti hathouti 293 févr. 21 14:13 long.txt
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex4$ ls -l | sort -n -k 8
total 12
-rw-rw-r-- 1 hathouti hathouti 293 févr. 21 14:13 long.txt
-rw-rw-r-- 1 hathouti hathouti 48 févr. 21 14:12 moyen.txt
-rw-rw-r-- 1 hathouti hathouti 6 févr. 21 14:12 court.txt
```

FIGURE 19 – `ls -l` puis `ls -l | sort -n -k 5` (taille croissante) et `ls -l | sort -n -k 8` (ordre alphabétique du nom)

Avec `ls -l | sort -n -k 5`, les fichiers sont triés par taille croissante : `court.txt` (6 octets), `moyen.txt` (48 octets), `long.txt` (293 octets). En changeant la colonne à `-k 8` (nom du fichier), le tri se fait alphabétiquement par nom dans l'ordre inverse.

4.2 Comptage des objets du répertoire avec `ls | wc -w`

On utilise `ls` combiné à `wc -w` pour compter le nombre de fichiers présents dans le répertoire courant :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex4$ ls
court.txt long.txt moyen.txt
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex4$ ls | wc -w
3
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex4$
```

FIGURE 20 – `ls | wc -w` : le répertoire `ex4` contient 3 fichiers

`ls` liste les fichiers, et `wc -w` compte les mots dans cette liste, ce qui correspond au nombre de fichiers. Le résultat est bien 3, correspondant aux trois fichiers créés.

5 Exercice 5 : Tâches de fond

Les commandes `bg`, `fg` et l'opérateur `&` permettent de gérer les processus en avant-plan et en arrière-plan depuis un même terminal.

5.1 Lancement de `gedit` sans `&` : blocage du terminal

On lance `gedit` sans l'opérateur `&` depuis un terminal vide :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ cd ex5
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex5$ ls
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex5$ gedit
```

FIGURE 21 – `gedit` lancé sans `&` : le terminal est bloqué, on ne peut plus y taper de commandes

On peut utiliser `gedit` normalement et y saisir du texte, mais le terminal est entièrement occupé par le processus : il est impossible d'y exécuter d'autres commandes tant que `gedit` est ouvert.

5.2 Suspension avec `Ctrl+Z`, reprise avec `bg` et `fg`

On lance à nouveau `gedit` sans `&`, puis on appuie sur `Ctrl+Z` pour suspendre le processus :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex5$ gedit
^Z
[1]+  Arrêté                  gedit
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex5$
```

FIGURE 22 – `Ctrl+Z` : le processus `gedit` est suspendu (`[1]+ Arrêté`), le terminal récupère la main

`gedit` est suspendu : la fenêtre graphique ne répond plus, comme en témoigne le message système :

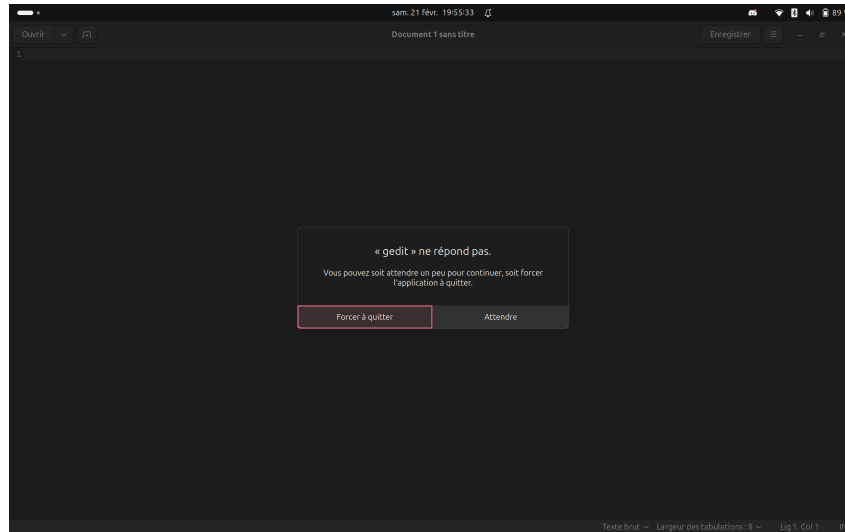


FIGURE 23 – `gedit` suspendu : le système signale que l'application ne répond pas

On tape `bg` dans le terminal pour relancer `gedit` en arrière-plan. Le processus reprend vie et la fenêtre redevient utilisable, tout en laissant le terminal libre :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex5$ gedit
^Z[1]  Processus arrêté      gedit

[2]+  Arrêté                  gedit
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex5$ bg
[2]+  gedit &
```

FIGURE 24 – `bg` : `gedit` reprend en arrière-plan (`[2]+ gedit &`), le terminal reste disponible

On peut alors saisir du texte dans `gedit` pendant que le terminal reste opérationnel :

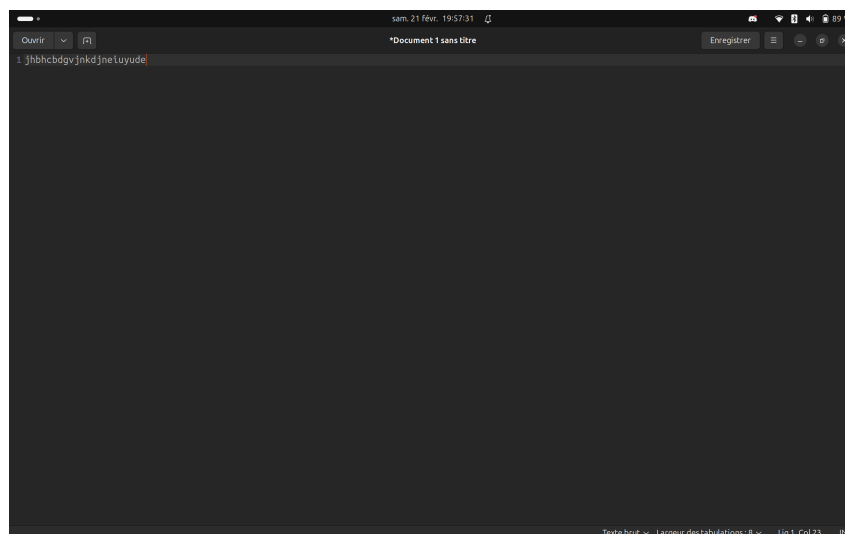


FIGURE 25 – `gedit` actif en arrière-plan : saisie de texte possible pendant que le terminal est libre

On utilise ensuite `fg` pour ramener `gedit` en avant-plan, ce qui re-bloque le terminal comme dans la situation initiale :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex5$ fg
gedit
```

FIGURE 26 – `fg` : `gedit` revient en avant-plan, le terminal est à nouveau bloqué

5.3 Lancement direct avec `gedit &`

La solution la plus pratique est de lancer `gedit` directement en arrière-plan avec `&`, ce qui évite de bloquer le terminal dès le départ :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex5$ gedit &
[1] 183074
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex5$ ls
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2/ex5$
```

FIGURE 27 – `gedit &` : le PID est affiché ([1] 183074), le terminal reste immédiatement disponible

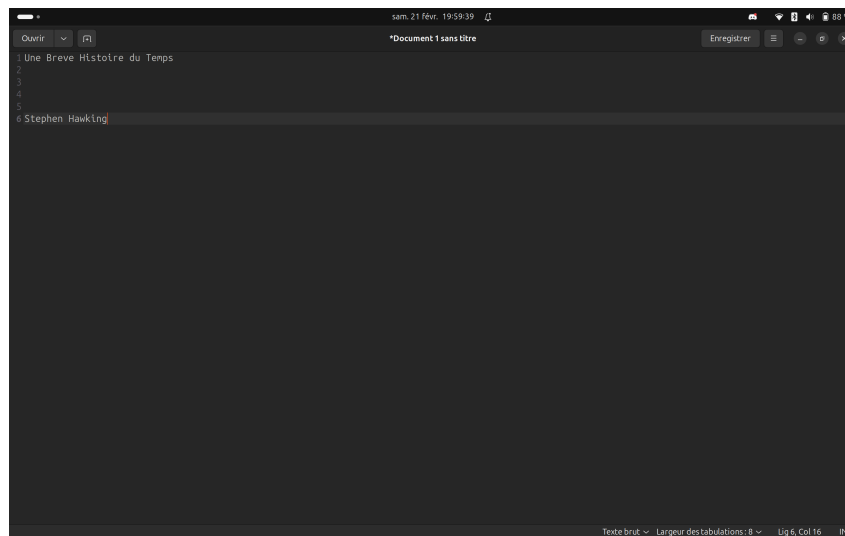


FIGURE 28 – `gedit` ouvert en arrière-plan avec du texte saisi, pendant que le terminal reste libre

C'est la méthode la plus efficace : `gedit &` permet d'utiliser simultanément l'éditeur graphique et le terminal sans avoir à gérer la suspension et la reprise du processus.

6 Exercice 6 : Alias

Un alias est un raccourci permettant d'associer un nom personnalisé à une commande. Il est créé avec `alias nom=commande` et supprimé avec `unalias nom`.

6.1 Création d'un alias temporaire

On crée l'alias `toto=ls` dans le terminal courant, puis on vérifie qu'il fonctionne correctement :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ alias toto=ls
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ ls
ex1 ex2 ex3 ex4 ex5 Screens TP2.pdf
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ toto
ex1 ex2 ex3 ex4 ex5 Screens TP2.pdf
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$
```

FIGURE 29 – alias `toto=ls` : la commande `toto` se comporte désormais comme `ls`

Après fermeture du terminal et ouverture d'un nouveau, l'alias a disparu. La commande `toto` n'est plus reconnue :

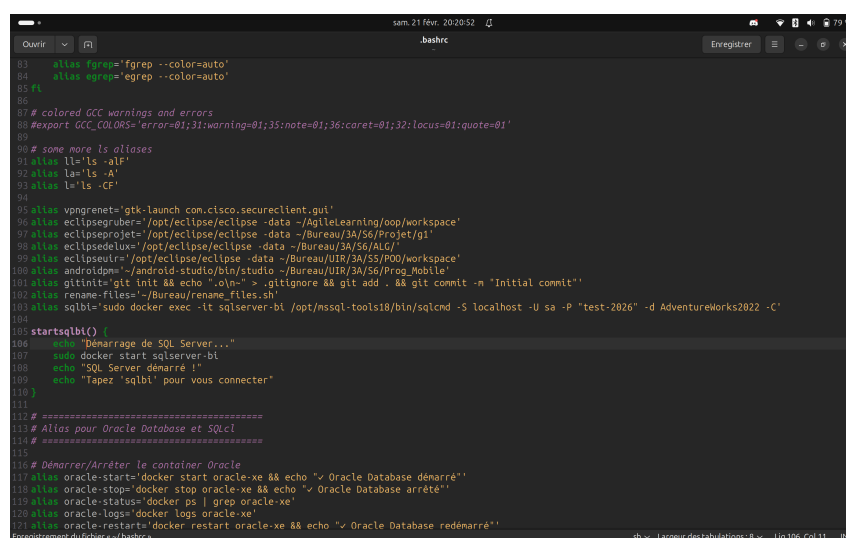
```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ ls
ex1 ex2 ex3 ex4 ex5 Screens TP2.pdf
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ toto
La commande « toto » n'a pas été trouvée, mais peut être installée avec :
sudo snap install toto
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$
```

FIGURE 30 – Nouveau terminal : `toto` n'est plus défini, l'alias temporaire n'a pas persisté

Un alias créé en ligne de commande est **temporaire** : il est propre à la session courante et disparaît à la fermeture du terminal.

6.2 Alias permanents via `~/.bashrc`

Pour rendre un alias permanent, il faut l'ajouter dans le fichier `~/.bashrc`, qui est exécuté automatiquement à chaque démarrage d'un nouveau shell. On ouvre le fichier avec `gedit ~/.bashrc` et on y ajoute les alias souhaités :



```
#!/bin/bash
# ~/.bashrc: executed by bash(1) when login shell is not bash(1)

# Aliases
alias fgrep='fgrep --color=auto'
alias egrep='egrep --color=auto'
alias fl='ls -la'
# colored GCC warnings and errors
export GCC_COLORS='error=01;31:warning=01;35:note=01;36:caret=01;32:locus=01;32:quote=01'
# some more ls aliases
alias la='ls -A'
alias l='ls -CF'
alias vpgnrenet='gtk-launch com.cisco.secureclient.gui'
alias eclipsegruber='/opt/eclipse/eclipse -data ~/AgileLearning/loop/workspace'
alias eclipseprojet='/opt/eclipse/eclipse -data ~/Bureau/3A/S6/Projet/g1'
alias eclipsedelux='/opt/eclipse/eclipse -data ~/Bureau/3A/S6/ALG/'
alias eclipseu2='/opt/eclipse/eclipse -data ~/Bureau/UIR/3A/S6/POO/workspace'
alias androidom~/android-studio/bin/studio ~/Bureau/UIR/3A/S6/Prog_Mobile'
alias gitinit='git init && echo ".gitignore" > .gitignore && git add . && git commit -m "Initial commit"'
alias rename-files~/Bureau/rename_files.sh'
alias sqldb='sudo docker exec -it sqlserver-bi /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P "test:2026" -d AdventureWorks2022 -C'
alias startsqlbi() {
    echo "Démarrage de SQL Server..."
    docker start sqlserver-bi
    echo "SQL Server démarré !"
    echo "Tapez 'sqlbi' pour vous connecter"
}
# =====
# Alias pour Oracle Database et SQLcl
# =====
# Démarrer/Arrêter le container Oracle
alias oracle-start='docker start oracle-xe && echo "Oracle Database démarré"'
alias oracle-stop='docker stop oracle-xe && echo "Oracle Database arrêté"'
alias oracle-status='docker ps | grep oracle-xe'
alias oracle-logs='docker logs oracle-xe'
alias oracle-restart='docker restart oracle-xe && echo "Oracle Database redémarré"'
# =====
```

FIGURE 31 – Contenu du fichier `~/.bashrc` : les alias permanents déjà présents dans le système

On applique les modifications sans redémarrer le terminal grâce à la commande `source` :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ gedit ~/.bashrc
^C
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ source ~/.bashrc
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$
```

FIGURE 32 – `source ~/.bashrc` : rechargement du fichier de configuration pour activer les nouveaux alias

Tout alias présent dans `~/.bashrc` sera disponible dans chaque nouveau terminal, contrairement aux alias temporaires créés directement en ligne de commande.

7 Exercice 7 : Variable PATH

La variable d'environnement `$PATH` contient la liste des répertoires dans lesquels le shell cherche les exécutables lorsqu'on tape une commande. Ces répertoires sont séparés par des `:`. La commande `type` permet de savoir où se trouve une commande donnée, ou si elle est définie comme alias.

7.1 Affichage de `$PATH` et localisation de commandes avec `type`

On affiche le contenu de la variable `$PATH` et on utilise `type` sur plusieurs commandes pour observer comment le shell les résout :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ echo "$PATH"
/home/hathouti/lv6-bin-dist/bin:/home/taha/Bureau/3A/ArM/lv6-bin-dist/bin:/home/hathouti/lv6-bin-dist/bin:/home/taha/Bureau/3A/ArM/lv6-bin-dist/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin:/home/hathouti/Android/Sdk/platform-tools:/home/hathouti/Android/Sdk/tools
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ type gedit
gedit est /usr/bin/gedit
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ type ls
ls est un alias vers « ls --color=auto »
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ type chrome
bash: type: chrome : non trouvé
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ type firefox
firefox est /usr/bin/firefox
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$
```

FIGURE 33 – `echo "$PATH"`, puis `type gedit`, `type ls`, `type chrome` et `type firefox`

On observe plusieurs cas :

- `type gedit` → `gedit est /usr/bin/gedit` : le shell a trouvé l'exécutable dans `/usr/bin`;
- `type ls` → `ls est un alias vers « ls --color=auto »` : `ls` est en réalité un alias défini dans `~/.bashrc`;
- `type chrome` → `non trouvé` : aucun exécutable nommé `chrome` n'existe dans les répertoires du `$PATH`;
- `type firefox` → `firefox est /usr/bin/firefox` : l'exécutable est bien présent dans `/usr/bin`.

7.2 Ajout d'une variable dans `~/.bashrc`

On ouvre `~/.bashrc` avec `gedit` et on y ajoute la ligne `dir1=/usr/include`. On recharge ensuite le fichier avec `source` et on vérifie la valeur de la variable :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ gedit ~/.bashrc
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ source ~/.bashrc
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ echo $dir1
/usr/include
hathouti@Taha-inspiron-16:~/Bureau/UIR/3A/S6/Prog_Syst/TP2$ █
```

FIGURE 34 – `source ~/.bashrc` puis `echo $dir1` : la variable affiche bien `/usr/include`

La variable `dir1` est maintenant définie de manière permanente. À chaque nouveau terminal, `echo $dir1` retournera `/usr/include` sans avoir besoin de la redéfinir manuellement.

Conclusion

Ce TP2 a permis d’approfondir la compréhension des mécanismes fondamentaux de la programmation système sous Linux, en explorant la gestion des processus, la redirection des entrées/sorties, les pipes et la personnalisation de l’environnement shell.

À travers les différents exercices, nous avons pu observer que chaque processus est une entité indépendante identifiée par un PID unique, et que la commande `ps` seule se limite au TTY courant tandis que `ps -u` ou `ps aux` offrent une vue globale sur l’ensemble des processus du système. Nous avons également compris la distinction entre `SIGTERM` et `SIGKILL`, le premier laissant au processus la possibilité de se terminer proprement, le second forçant sa terminaison immédiate par le noyau.

La redirection des flux et les pipes se sont révélés être des outils puissants permettant de chaîner des commandes et de manipuler les données efficacement, sans recours à des fichiers intermédiaires inutiles. La gestion des tâches de fond avec `&`, `bg` et `fg` offre quant à elle une flexibilité précieuse pour travailler sur plusieurs processus depuis un même terminal.

Enfin, la personnalisation du shell via les alias permanents dans `~/.bashrc` et la maîtrise de la variable `$PATH` constituent des bases essentielles pour tout utilisateur souhaitant optimiser son environnement de travail sous Linux.